

JavaScript from zero

How to make your web page respond when someone clicks, types or votes.

```
let naija = 0;

button.addEventListener("click", function () {
  naija = naija + 1;
  score.textContent = `Naija ${naija}`;
});
```

How to use these notes

1. Read the short explanation first.
2. Type the example yourself. Do not copy and paste it; typing it out is how your fingers learn where the brackets go.
3. Run it and check that your console shows what the **Prints** box says.
4. Do the **Try it** task without looking back at the example. Look back only if you get stuck.

The red **Watch out** boxes are the mistakes that catch almost everybody. Read them even when your code works.

1 Where JavaScript goes

2 Values and variables

3 Text, numbers and the + trap

4 Functions

5 Making decisions

6 Lists and loops

7 Objects

8 Changing the page

9 Responding to clicks and typing

10 When it breaks

11 Cheat sheet

12 Practice at home

Where JavaScript goes

Your page already has two files. HTML says what things are: a heading, a button, a paragraph. CSS says how they look. JavaScript is the third file, and it decides what happens when someone clicks, types or scrolls.

Link it at the bottom of the body, just before `</body>`. The browser reads your page from top to bottom, so putting the script last means every element already exists by the time your code runs.

index.html

```
<footer>Made in Lagos</footer>
<script src="script.js"></script>
</body>
</html>
```

To see what your code is doing, open the console. Press **F12** and click the Console tab. Anything you pass to `console.log` shows up there.

script.js

```
console.log("Hello from Lagos");
console.log(2 + 2);
```

Prints

Hello from Lagos

4

Watch out

Nothing in the console? Check that the file name in `src` matches the real file exactly (`script.js` is not `Script.js`), that both files sit in the same folder, and that you saved before refreshing.

Try it

Log your name and your state of origin on two separate lines.

Values and variables

Almost everything you work with is one of three kinds of value. A **number** has no quotes and you can do sums with it: `1500`, `0`, `-20`. A **string** is text inside quotes: `"Chiamaka"`, `"Yaba"`. A **boolean** is only ever `true` or `false`.

A variable is a name you give a value so you can use it again later.

script.js

```
const name = "Chiamaka";    // string
let dataLeft = 1500;        // number: MB of data left
const hasLight = true;      // boolean

dataLeft = dataLeft - 200;  // after watching football highlights

console.log(name, dataLeft, hasLight);
```

Prints

Chiamaka 1300 true

Use `const` by default. Switch to `let` only when the value will change, like a data balance that keeps going down.

Read `=` as *gets*. The line `dataLeft = dataLeft - 200` means "dataLeft gets dataLeft minus 200". JavaScript works out the right side first, then stores the answer in the name on the left.

Give variables names that explain themselves: `dataLeft`, not `d` or `x`. Start with a small letter, use no spaces, and put a capital letter where each new word starts.

Watch out

Change a `const` and the console says *Assignment to constant variable*. That message tells you exactly what happened. If the value really needs to change, use `let`.

Try it

Make a variable for your airtime balance, spend ₦100 on a call, then log what is left.

03

Text, numbers and the + trap

Join text with backticks. Anything inside `${ }` gets worked out and dropped into the sentence. On most keyboards the backtick key sits to the left of the 1 key.

script.js

```
const customer = "Tunde";
const total = 3500;

console.log(`Thank you ${customer}, your total is ₦${total}.`);
```

Prints

Thank you Tunde, your total is ₦3500.

Now the trap. The + sign adds numbers, but if either side is text it glues them together instead. Anything a person types into a form reaches your code as text, even when it looks like a number.

script.js

```
const amount = "2000"; // typed into a form
const charge = 50;

console.log(amount + charge); // glued
console.log(Number(amount) + charge); // added
```

Prints

200050

2050

Wrap text in `Number()` before you do sums with it.

Some numbers should stay as text. You will never add two phone numbers together, and if you turn one into a number the zero at the front disappears:

script.js

```
console.log(Number("08031234567"));
```

Prints

8031234567

To check what kind of value you have, use `typeof`:

script.js

```
console.log(typeof "5"); // string
console.log(typeof 5); // number
console.log(typeof false); // boolean
```

Watch out

`${ }` only works inside backticks. Inside ordinary quotes you get `${customer}` printed exactly as you typed it.

Try it

Print *Aisha scored 280 in JAMB* using one variable for the name and one for the score.

Functions

A function is a set of steps with a name. You write the steps once, then run them as many times as you need, with different inputs each time.

script.js

```
function greet(name) {
  console.log(`Good morning, ${name}`);
}

greet("Emeka");
greet("Zainab");
```

Prints

```
Good morning, Emeka
Good morning, Zainab
```

Writing the function does nothing on its own. Think of a recipe in a drawer: nothing gets cooked until somebody uses it. The brackets in `greet("Emeka")` are what run it.

`name` is the **parameter**, a blank in the recipe. `"Emeka"` is the **argument**, whatever fills that blank this time.

return versus console.log

Most people get stuck here, so go slowly. `console.log` only shows you a value. `return` hands the value back, so the rest of your code can use it.

You and three friends eat at a buka and the bill comes to ₦12,000:

script.js

```
function splitBill(total, people) {
  return total / people;
}

const each = splitBill(12000, 4);
console.log(`Everybody pays ₦${each}`);
```

Prints

```
Everybody pays ₦3000
```

Here is the same function with `console.log` where `return` should be:

```
script.js

function splitBill(total, people) {
  console.log(total / people);
}

const each = splitBill(12000, 4); // prints 3000
console.log(each);                // prints undefined
```

The function showed you 3000 and then threw it away. A function without `return` always hands back `undefined`. To remember it: **console.log is a window, return is a door**. You look through a window. You carry things out through a door.

`return` also ends the function straight away. Any line after it never runs.

Watch out

`splitBill` on its own, without brackets, runs nothing. You need `splitBill(12000, 4)`.

Try it

Write `addDelivery(price)` that returns the price plus ₦1,500 for delivery. `addDelivery(8000)` should give you 9500.

05

Making decisions

`if` lets your code choose between two paths. The condition goes in round brackets, and the code for each path goes in curly braces.

```
script.js

const jambScore = 245;
const cutOff = 200;

if (jambScore >= cutOff) {
  console.log("You made the cut-off. Apply!");
} else {
  console.log("Below the cut-off for this school.");
}
```

Prints

You made the cut-off. Apply!

For more than two paths, add `else if`. JavaScript checks each condition in order and runs the first one that is true:

script.js

```
const hasLight = false;
const hasFuel = true;

if (hasLight) {
  console.log("Up NEPA! Charge everything.");
} else if (hasFuel) {
  console.log("Generator time.");
} else {
  console.log("Candle and early sleep.");
}
```

Prints

Generator time.

===	equal	!==	not equal
>	greater than	>=	greater than or equal to
<	less than	<=	less than or equal to

Always compare with three equals signs. Two equals signs change the types before comparing, and that causes surprises:

script.js

```
console.log("5" == 5); // true, surprising
console.log("5" === 5); // false, correct
```

Join conditions with `&&` (and) or `||` (or). Flip one with `!` (not):

script.js

```
const age = 16;
const hasPaid = true;

console.log(age >= 13 && age <= 19); // true: a teenager
console.log(hasPaid || age < 12); // true: one of them is true
console.log(!hasPaid); // false
```

Watch out

One `=` puts a value into a variable. `===` compares two values. `if (score = 200)` is a bug that looks almost right.

Try it

Make a variable `dataLeft`. If it is below 100, log a warning to buy data. Otherwise, log that you are fine for now. Write the messages in Pidgin if you like.

06

Lists and loops

An array keeps a list of values under one name. Use square brackets, with commas between the items.

```
script.js

const stops = ["Ojota", "Maryland", "Anthony"];

console.log(stops[0]);    // Ojota
console.log(stops.length); // 3

stops.push("Palmgrove");
console.log(stops.length); // 4
```

Counting starts at 0, so `stops[0]` is the first stop. The last one is always `stops[stops.length - 1]`.

A loop runs the same code once for every item in the list. Learn `for...of` first:

```
script.js

for (const stop of stops) {
  console.log(`${stop}! ${stop}! Enter with your change.`);
}
```

Prints

```
Ojota! Ojota! Enter with your change.
Maryland! Maryland! Enter with your change.
Anthony! Anthony! Enter with your change.
Palmgrove! Palmgrove! Enter with your change.
```

Add a fifth stop to the array and the loop picks it up without any other change. You write the instruction once, however long the list gets.

Adding up a list

Make the total before the loop starts, then add to it inside the loop:

script.js

```
const prices = [2000, 1500, 500]; // jollof, chicken, zobo
let total = 0;

for (const price of prices) {
  total = total + price;
}

console.log(`Your bill: ₦${total}`);
```

Prints

Your bill: ₦4000

Two loops you will see in other people's code

script.js

```
for (let i = 1; i <= 3; i++) {
  console.log(`Round ${i}`);
}

stops.forEach(function (stop) {
  console.log(stop);
});
```

Both work. Stick with `for...of` in your own code until you have a reason not to.

Watch out

`stops[4]` on a list of four items gives `undefined`, not an error. If a value is mysteriously `undefined`, check whether you counted from 1 instead of 0.

Try it

Make a list of five test scores. Log how many there are, the first and the last score, every score on its own line, and the total. Then work out the average.

Objects

An array numbers its items. An object names them. Use an object when one thing has several details.

script.js

```
const student = {
  name: "Ifeoma",
  state: "Enugu",
  jambScore: 268,
  hasLaptop: true
};

console.log(student.name);      // Ifeoma
console.log(student.jambScore); // 268

student.hasLaptop = false;      // change a value
```

Read a value with a dot and the property name. Change it the same way.

A list of objects

Put objects inside an array and you have the shape of most real data: a menu, a class list, a playlist, a bank statement.

script.js

```
const menu = [
  { food: "Jollof rice", price: 2000 },
  { food: "Fried plantain", price: 700 },
  { food: "Pepper soup", price: 2500 }
];

const budget = 2000;

for (const item of menu) {
  if (item.price <= budget) {
    console.log(`You can afford ${item.food}`);
  }
}
```

Prints

```
You can afford Jollof rice
You can afford Fried plantain
```

That one loop uses an array, objects, a loop, a comparison and a template literal. Most of what you build from here is some mix of those.

Watch out

Write `student.name`. Not `student."name"`, and not `student[name]`. A dot, then the property name, no quotes.

Try it

Model four players from your favourite team as objects with a name, a position and a number of goals. Loop through them and log only the players with more than 2 goals.

08

Changing the page

When the browser loads your HTML, it turns every element into an object your code can grab and change. `document.querySelector` finds one element, using the same selectors you already write in CSS.

index.html

```
<h1 class="shop-name">Mama Put</h1>
<p class="status">Checking...</p>
```

script.js

```
const shopName = document.querySelector(".shop-name");
const status = document.querySelector(".status");

shopName.textContent = "Iya Basira Kitchen";
status.textContent = "Open now. Jollof is ready.";
status.classList.add("open");
```

A few properties do most of the work:

<code>textContent</code>	The words inside an element. Read it, or replace it.
<code>classList.add("x")</code> <code>classList.remove("x")</code>	Switch a CSS class on or off. Keep the styling in your CSS file and let JavaScript flip the class.
<code>style.color</code>	Set one style directly. Handy while testing, but classes are neater.

`querySelector` gives you the first match. `querySelectorAll` gives you every match, as a list:

script.js

```
const items = document.querySelectorAll(".menu li");
console.log(items.length);
```

Watch out

Cannot read properties of null means `querySelector` found nothing. Look for a typo in the selector (`.stauts`), a missing dot before a class name, or a script tag that sits above the element it is looking for.

Try it

On your own page, change the main heading and add a class to one element, using only JavaScript. Do not touch the HTML file.

09

Responding to clicks and typing

`addEventListener` tells an element: when this happens, run that function.

index.html

```
<button class="vote-naija">Naija jollof</button>
<button class="vote-ghana">Ghana jollof</button>
<p class="score">No votes yet</p>
```

script.js

```
let naija = 0;
let ghana = 0;

const score = document.querySelector(".score");

function showScore() {
  score.textContent = `Naija ${naija} : ${ghana} Ghana`;
}

document.querySelector(".vote-naija").addEventListener("click", function () {
  naija = naija + 1;
  showScore();
});

document.querySelector(".vote-ghana").addEventListener("click", function () {
  ghana = ghana + 1;
  showScore();
});
```

Both buttons call the same `showScore` function, so the paragraph always reads the same way. Make that a habit: one function updates the page, and everything that changes something calls it.

Other events follow the same shape. Only the word in quotes changes: `"input"` fires on every key pressed in a text box, and `"submit"`, `"keydown"` and `"mouseover"` work the same way.

The brackets bug

script.js

```
button.addEventListener("click", showScore()); // wrong
button.addEventListener("click", showScore);   // right
```

With brackets, `showScore` runs once, straight away, and the button does nothing after that. Without brackets, you hand the function over so it can run on every click.

Reading what someone typed

The text in an input box lives in `.value`, and it is always a string.

index.html

```
<input class="bill" placeholder="Bill total">
<input class="people" placeholder="Number of people">
<button class="split">Split it</button>
<p class="answer"></p>
```

script.js

```
document.querySelector(".split").addEventListener("click", function () {
  const bill = Number(document.querySelector(".bill").value);
  const people = Number(document.querySelector(".people").value);

  document.querySelector(".answer").textContent =
    `Each person pays N${bill / people}`;
});
```

Watch out

Dividing text happens to work without `Number()`. Adding does not: `"2000" + 50` is still `"200050"`. Use `Number()` on every `.value` and you never have to remember which is which.

Try it

Build a light switch. A button flips a variable `hasLight` between `true` and `false` (`hasLight = !hasLight` does it). When it is true, a paragraph reads *Up NEPA! Light don come*. When it is false, it reads *Light don go again*. Put the `if` inside your update function.

10

When it breaks

Every programmer reads error messages all day. The message tells you what went wrong and which line to look at.

1. Read the red error in the console. All of it, including the line number.
2. Say out loud what you expected, and what actually happened.
3. `console.log` any value you are not sure about.
4. Change one thing, save, refresh and check again.
5. Do not delete your work and start again. Starting over hides the mistake instead of teaching you what it was.

Errors you will see this week

The console says	It usually means
<code>jolof is not defined</code>	A typo in a name, or you used the name before creating it.
<code>Cannot read properties of null</code>	<code>querySelector</code> found nothing.
<code>Assignment to constant variable</code>	You tried to change a <code>const</code> .
<code>Unexpected token</code>	A missing bracket, brace, comma or quote nearby.
<code>showScore is not a function</code>	A typo in the name, or the thing you called is not a function.
No error, but it is still wrong	
200050 instead of 2050	You added text and a number. Use <code>Number()</code> .
A function gives back <code>undefined</code>	It uses <code>console.log</code> where it needs <code>return</code> .
The button does nothing	You passed <code>showScore()</code> instead of <code>showScore</code> , or the selector is wrong.

Cheat sheet

values and decisions

```
// values
const name = "Chiamaka";
let balance = 1500;
const isOpen = true;

// join text
`Hi ${name}, you have ₦${balance}`

// text to number
Number("2000")

// decide
if (balance > 1000) {
  // ...
} else {
  // ...
}

// compare    ===  !==  >  <  >=  <=
// combine    &&  ||  !
```

functions, lists and objects

```
// function
function addDelivery(price) {
  return price + 1500;
}

// list
const stops = ["Ojota", "Maryland"];
stops[0]
stops.length
stops.push("Yaba");

// loop
for (const stop of stops) {
  console.log(stop);
}

// object
const student = {
  name: "Ifeoma",
  score: 268
};
student.score
```

the page

```
// find something on the page
const status = document.querySelector(".status");

// change it
status.textContent = "Open now";
status.classList.add("open");
status.classList.remove("closed");

// run code when something happens
button.addEventListener("click", function () {
  // ...
});

// what someone typed is always a string, so use Number() before sums
const amount = Number(input.value);
```

Practice at home

Each project uses only what is in these notes. Build them in a new folder with an `index.html` and a `script.js`.

01 Data balance tracker

Start with 2000 MB. One button spends 150 (watching a video), one spends 5 (sending a WhatsApp message), and one adds 1000 (buying data). Show the balance on the page. When it drops below 200, add a warning class to it.

02 Bill splitter

Build the splitter from section 9, then add a third box for a delivery fee that gets added to the bill before it is split.

03 Jollof war

Take the vote counter from section 9. Add a Reset button and a line that says who is winning, or that it is a draw.

04 Cut-off checker

Two boxes: a JAMB score and a school's cut-off mark. A button tells the student whether they made it. If either box is empty, show a message asking them to fill it in.

05 Buka menu

An array of at least five meals as objects, each with a name and a price. A budget box and a button that logs every meal you can afford, then the cheapest meal on the menu.

Pick one, bring it to the next class working, and be ready to explain every line.